

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Лабораторная работа №2
на тему
«Конфигурационное пространство PCI»

Выполнил:

Кравченко М.Д.

Проверил:

Селезнёв А.И.

МИНСК 2025

ВВЕДЕНИЕ

Шина PCI и её современные модификации являются ключевым компонентом архитектуры современных вычислительных систем, обеспечивая подключение критически важных устройств от видеокарт до сетевых контроллеров. От корректной работы PCI-устройств напрямую зависит производительность и стабильность компьютерной системы в целом.

Современные операционные системы, включая Windows, предоставляют развитые API для анализа подключенного оборудования. В данной работе основное внимание уделяется использованию Windows SetupAPI для получения информации о PCI-устройствах системы.

Целью работы является разработка программного модуля для обнаружения PCI-устройств и отображения их характеристик в удобном для пользователя виде. Для этого необходимо:

- изучить принципы работы Windows SetupAPI;
- разработать механизм извлечения идентификаторов VendorID и DeviceID;
- реализовать преобразование технических ID в понятные названия;
- создать систему классификации устройств;
- интегрировать функционал в графическое приложение.

Выполнение работы позволяет получить практические навыки работы с системными API Windows и разработки прикладных решений на C++/Qt, что важно для подготовки специалистов в области системного программирования.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Определение и характеристики шины PCI

Шина PCI (Peripheral Component Interconnect) представляет собой стандарт системной шины для подключения периферийных устройств к материнской плате компьютера. Характеризуется параллельной передачей данных, разрядностью 32 или 64 бита, тактовой частотой 33-66 МГц и пропускной способностью до 533 МБ/с. Поддерживает технологию Plug and Play для автоматической конфигурации устройств и арбитраж для управления доступом к шине.

1.2 Архитектура ЭВМ

Архитектура ЭВМ описывает организацию и взаимосвязь основных компонентов вычислительной системы: процессора, памяти, устройств ввода-вывода и системных шин. Основана на принципах фон Неймана, включая разделение памяти для данных и команд, последовательное выполнение инструкций и двоичное кодирование информации. Современные архитектуры предусматривают иерархию памяти, конвейеризацию и параллельную обработку данных.

1.3 Критерии и классификация периферийных устройств

Периферийные устройства классифицируются по функциональному назначению на устройства ввода, вывода, хранения данных и коммуникационные. Критериями классификации служат способ подключения, тип интерфейса, скорость передачи данных и назначение устройства. Устройства могут быть внутренними и внешними, подключаемыми через различные интерфейсы с разной пропускной способностью и протоколами обмена.

1.4 Определение понятия “Шина”

Шина представляет собой систему соединений и протоколов, обеспечивающую передачу данных между компонентами компьютера. Состоит из трех основных компонентов: шины данных для передачи информации, шины адреса для указания местоположения и шины управления для координации обмена. Шины характеризуются пропускной способностью, разрядностью и протоколом arbitration для разрешения конфликтов доступа.

1.5 Параметры и иерархия шин

Параметры шин включают разрядность, тактовую частоту, пропускную способность и тип передачи данных. Иерархия шин в современных системах включает процессорную шину, шину памяти, шину ввода-вывода высокого уровня и низкоскоростные шины периферии. Каждый уровень иерархии оптимизирован под определенные задачи и скорости передачи данных, обеспечивая баланс между производительностью и стоимостью.

1.6 Топология интерфейсов

Топология интерфейсов определяет физическую и логическую организацию соединений между устройствами. Основные типы включают общую шину, звездообразную, кольцевую и смешанную топологии. Выбор топологии влияет на производительность, надежность и масштабируемость системы. В компьютерных системах преобладают иерархические топологии с централизованным управлением

1.7 Архитектура и топология шины PCI

Архитектура шины PCI основана на параллельной передаче данных по общей шине с централизованным арбитражем. Топология представляет собой древовидную структуру с мостами, позволяющую создавать несколько сегментов шины. Шина использует мультиплексирование адреса и данных, поддерживает конфигурационное пространство для идентификации устройств и их параметров.

1.8 Доступ к шине PCI и фазы транзакции

Доступ к шине PCI регулируется арбитром, который предоставляет право управления шиной устройствам. Транзакция состоит из фаз: арбитража, адресации, передачи данных и завершения. В фазе арбитража устройство запрашивает контроль над шиной. Фаза адресации определяет тип операции и адрес устройства. Фаза передачи данных может включать несколько циклов для пакетной передачи.

1.9 Режимы кэширования шины PCI

Режимы кэширования шины PCI включают write-through и write-back политики. Write-through обеспечивает одновременную запись в кэш и основную память, гарантируя согласованность данных. Write-back откладывает запись в память, повышая производительность. Шина

поддерживает протоколы кэш-когерентности для многопроцессорных систем, используя сигналы snooping для отслеживания состояния кэш-линий.

1.10 Шина PCI-X

Шина PCI-X представляет собой эволюционное развитие стандарта PCI с увеличенной тактовой частотой до 133 МГц и улучшенным протоколом передачи. Сохраняет обратную совместимость с PCI устройствами, но обеспечивает пропускную способность до 1,06 ГБ/с. Улучшения включают отдельные транзакции, уменьшенные латентности и более эффективный арбитраж, что особенно важно для серверных применений.

1.11 Шина PCI-Express

Шина PCI-Express представляет собой последовательную шину с архитектурой point-to-point, заменяющую параллельные шины PCI и PCI-X. Использует дифференциальные сигналы и packet-based протокол передачи. Масштабируется от x1 до x32 линий, где каждая линия обеспечивает дуплексную передачу. Пропускная способность PCIe 4.0 достигает 64 ГБ/с для конфигурации x16, поддерживая горячее подключение и качество обслуживания.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Код программы

Файл MainProject.pro

```
QT       += core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
CONFIG += c++17
QT       += widgets
LIBS     += -lPowerProf
LIBS     += -lsetupapi
SOURCES += \
    animationplayer.cpp \
    batterymonitor.cpp \
    main.cpp \
    mainwindow.cpp \
    pcideviceswidget.cpp
HEADERS  += \
    animationplayer.h \
    animations.h \
    batterymonitor.h \
    mainwindow.h \
    pcideviceswidget.h
FORMS    += \
    mainwindow.ui
qnx: target.path = /tmp/${TARGET}/bin
else: unix:!android: target.path = /opt/${TARGET}/bin
!isEmpty(target.path): INSTALLS += target
RESOURCES += \
    img.qrc
DISTFILES += \
    resources/img/Background/Background_main4k.png
```

Файл main.cpp

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;

    w.setFixedSize(1100, 700);
    w.setWindowTitle("Grimm present");

    QIcon iconApp(":/logo/resources/img/logo/logo.ico");
    w.setWindowIcon(iconApp);

    w.show();
    return a.exec();
}
```

Файл mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QTimer>
#include <QPixmap>
#include <QVector>
#include "animationplayer.h"
#include "batterymonitor.h"
#include <QPushButton>
#include <QLabel>
#include <QPropertyAnimation>
#include <QMap>
#include "pcideviceswidget.h"
#include <QMainWindow>
QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE
class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void onPageChanged(int index); // Когда происходит смена окон
    void updateBatteryUI(); // слот для обновления UI батареи
    void updateChargingAnimation(bool charging); // Обновление анимац при смене тпа питания
}
```

```
private:
    Ui::MainWindow *ui;
    AnimationPlayer* characterAnim;
    AnimationPlayer* characterL1Anim;
    AnimationPlayer* characterL2Anim;
    AnimationPlayer* backgroundAnim;
    BatteryMonitor* batteryMonitor;
    // Метки и кнопки для Батареи
    QLabel* powerSourceLabel;
    QLabel* batteryLevelLabel;
    QLabel* powerSavingLabel;
    QLabel* batteryTimeLabel;
    QPushButton* sleepBtn;
    QPushButton* hibernateBtn;
    QMap<QWidget*, QRect> m_savedGeometries;
    PciDevicesWidget *pciDevicesWidget;
};
#endif
```

Файл mainwindow.cpp

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "animationplayer.h"
#include "animations.h"
#include "iostream"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    // ----- Инициализация Мониторинга Батареи -----
    batteryMonitor = new BatteryMonitor(this);

    // Подвязываем
    powerSourceLabel = ui->PowerSourceLabel;
    batteryLevelLabel = ui->BatteryLevelLabel;
    powerSavingLabel = ui->PowerSavingModelLabel;
    batteryTimeLabel = ui->BatteryTimeLabel;
    sleepBtn = ui->SleepButton;
    hibernateBtn = ui->HibernateButton;

    // Обновляем UI при изменении статуса батареи
    connect(batteryMonitor, &BatteryMonitor::statusChanged, this, &MainWindow::updateBatteryUI);
    connect(batteryMonitor, &BatteryMonitor::chargingChanged, this, &MainWindow::updateChargingAnimation);

    // Кнопки сна и гибернации
    connect(sleepBtn, &QPushButton::clicked, batteryMonitor, &BatteryMonitor::sleep);
    connect(hibernateBtn, &QPushButton::clicked, batteryMonitor, &BatteryMonitor::hibernate);

    // Начальное обновление UI
    updateBatteryUI();

    // Инициализация виджета PCI
    pciDevicesWidget = new PciDevicesWidget(ui->PCITable, this);

    // Вставляем таблицу виджета в UI QTableWidgetItem
    // Можно просто использовать существующий QTableWidgetItem, заменяя внутренние данные
    //pciDevicesWidget->setParent(ui->PCITable);
    //pciDevicesWidget->resize(ui->PCITable->size());
    //pciDevicesWidget->show();

    connect(ui->stackedWidget, &QStackedWidget::currentChanged, this, &MainWindow::onPageChanged);
    this->setStatusBar(nullptr);

    //Устанавливаем курсор
    QCursor cursorTarget = QCursor(QPixmap(":/cursor/resources/coursore/coursore.png"), 0, 0);
    this->setCursor(cursorTarget);

    // ----- Анимации Main Window
    // Подключение анимации к label
    characterAnim = new AnimationPlayer(ui->CharacterLabel, this);
    backgroundAnim = new AnimationPlayer(ui->BackgroundLabel, this);

    characterAnim->setLoopAnimation(getLoopFrames());
    backgroundAnim->setLoopAnimation(getBackgroundFrames());

    // ----- Анимации L1 Window
    characterL1Anim = new AnimationPlayer(ui->GrimmL1, this);
    //Цикловая Анимация
    characterL1Anim->setLoopAnimation(getLoopFrames());

    // ----- Анимации L2 Window
    characterL2Anim = new AnimationPlayer(ui->GrimmL2, this);
    //Цикловая Анимация
    characterL2Anim->setLoopAnimation(getLoopFrames());

    // ----- Отдельные анимации
    //ПОКЛОН
    Animation reveranceAnim;
    reveranceAnim.frames = getReveranceFrames();
    reveranceAnim.loop = false;
    characterAnim->addAnimation("reverance", reveranceAnim);
    characterL1Anim->addAnimation("reverance", reveranceAnim);
    //ИНТРО
    Animation introAnim;
    introAnim.frames = getIntroFrames();
```

```

introAnim.loop = false;
characterAnim->addAnimation("intro", introAnim);
characterL1Anim->addAnimation("intro", introAnim);
characterL2Anim->addAnimation("intro", introAnim);
//БЭК
Animation backAnim;
backAnim.frames = getBackFrames();
backAnim.loop = false;
characterL2Anim->addAnimation("back", backAnim);
//ОУТРО
Animation outroAnim;
outroAnim.frames = getOutroFrames();
outroAnim.loop = false;
characterL1Anim->addAnimation("outro", outroAnim);
//ШЕЛЧЕК
Animation handsOutAnim;
handsOutAnim.frames = getHandsOutFrames();
handsOutAnim.loop = false;
characterAnim->addAnimation("handsOut", handsOutAnim);
characterL1Anim->addAnimation("handsOut", handsOutAnim);
//ИНТРО ЗАРЯДКА
Animation IntroChargingAnim;
IntroChargingAnim.frames = getIntroChargingFrames();
IntroChargingAnim.loop = false;
characterL1Anim->addAnimation("introCharging", IntroChargingAnim);
//ОУТРО ЗАРЯДКА
Animation OutroChargingAnim;
OutroChargingAnim.frames = getOutroChargingFrames();
OutroChargingAnim.loop = false;
characterL1Anim->addAnimation("outroCharging", OutroChargingAnim);
//Плещ
Animation CloackAnim;
CloackAnim.frames = getCloakFrames();
CloackAnim.loop = false;
characterAnim->addAnimation("cloakAnim", CloackAnim);

// ----- Запуск анимаций
characterAnim->playAnimation("intro");
characterAnim->start();
characterL1Anim->start();
characterL2Anim->start();
backgroundAnim->start();

// ----- ПЕРЕХОД МЕЖДУ ОКНАМИ-----
//Main->L1
connect(ui->Lab1BTN, &QPushButton::clicked, this, [this]() {
    characterAnim->playSequence({"reverance", "handsOut"}, [this]() {
        ui->stackedWidget->setCurrentIndex(2);
    });
});
//L1->Main
connect(ui->Back, &QPushButton::clicked, this, [=]() {
    characterL1Anim->playSequence({"reverance", "outro"}, [this]() {
        ui->GrimmL1->hide();
        QTimer::singleShot(500, this, [this]() {
            ui->stackedWidget->setCurrentIndex(0);
        });
    });
});
//Main->L2
connect(ui->Lab2BTN, &QPushButton::clicked, this, [this]() {
    // защитный указатель на лейбл
    QPointer<QLabel> label = ui->CharacterLabel;
    if (!label) return;

    // сохраним исходную геометрию (если ещё не сохранена)
    if (!m_savedGeometries.contains(label)) {
        m_savedGeometries[label] = label->geometry();
    }
    const QRect origGeom = m_savedGeometries[label];

    // вычислим новую геометрию (увеличение в 1.5 раза по центру)
    const qreal scale = 1.8; // можно поставить 3.0 если нужен больший zoom
    const int newW = static_cast<int>(origGeom.width() * scale);
    const int newH = static_cast<int>(origGeom.height() * scale);
    const QPoint center = origGeom.center();
    const QRect bigGeom(center.x() - newW/2, center.y() - newH/2, newW, newH);

    // Немедленно устанавливаем увеличенную геометрию (без анимации)
    label->setGeometry(bigGeom);

    // Просим цикл событий нарисовать обновление до старта анимации персонажа.
    // Вместо QApplication::processEvents() используем singleShot(0) – безопаснее.
    QTimer::singleShot(0, this, [this, label, origGeom]() {
        if (!label) {
            this->setEnabled(true);
            return;
        }

        // Теперь запускаем последовательность анимаций персонажа.
        // ВАЖНО: убедись, что имя анимации написано точно так же, как при добавлении.
        // В твоём коде есть getCloakFrames() – имя должно совпадать с тем, что ты добавляешь.
        characterAnim->playSequence({"cloakAnim"}, [this, label, origGeom]() {
            // Переключаем страницу (индекс ставь тот, который нужен)
            const int targetIndex = 1; // <- заменить в случае необходимости
            ui->stackedWidget->setCurrentIndex(targetIndex);

            // Восстанавливаем исходную геометрию сразу (без анимации)
            if (label) {
                label->setGeometry(origGeom);
            }
        });
    });
});

```



```

        // Включаем интерфейс обратно
        this->setEnabled(true);
    });
});
});
//L2->Main
connect(ui->L2backToMain, &QPushButton::clicked, this, [=]() {
    characterL2Anim->playSequence({"back"}, [this]() {
        ui->stackedWidget->setCurrentIndex(0);
        pciDevicesWidget->refreshDevices();
    });
});
}

MainWindow::~MainWindow()
{
    delete ui;
}

// Действия при смене страницы
void MainWindow::onPageChanged(int index)
{
    switch (index) {
        case 0: // Главная страница
            characterAnim->playAnimation("intro");
            break;
        case 2: // Вторая страница
            ui->BACK_BTNLabel->hide(); ui->Back->hide(); ui->BatteryLevelLabel->hide(); ui->BatteryTimeLabel->hide(); ui->
            >BatteryTypeLabel->hide();
            ui->GrimmL1->hide(); ui->HibernateButton->hide(); ui->Hibernate_BTNLabel_3->hide(); ui->Info_table->hide(); ui->
            >PowerSavingModeLabel->hide();
            ui->PowerSourceLabel->hide(); ui->SLEEP_BTNLabel_2->hide(); ui->SleepButton->hide();

            // 1 кадр
            ui->Background1Lab->setPixmap(QPixmap(":/bg/resources/img/Background/1Lab/Bg1frame.png"));

            // Через 1 сек второй кадр
            QTimer::singleShot(1000, this, [this]() {
                ui->Background1Lab->setPixmap(QPixmap(":/bg/resources/img/Background/1Lab/Bg2frame.png"));
            });

            // Через 1 сек персонажа
            QTimer::singleShot(1000, this, [this]() {
                ui->GrimmL1->show();

                // Стартовая анимация персонажа
                characterL1Anim->playSequence({"intro", "handsOut"}, [this]() {
                    ui->Background1Lab->setPixmap(QPixmap(":/bg/resources/img/Background/1Lab/backgroundClear4k.png"));
                    ui->BACK_BTNLabel->show(); ui->Back->show(); ui->BatteryLevelLabel->show(); ui->BatteryTimeLabel->show();
                    ui->BatteryTypeLabel->show(); ui->HibernateButton->show(); ui->Hibernate_BTNLabel_3->show();
                    ui->Info_table->show(); ui->PowerSavingModeLabel->show(); ui->PowerSourceLabel->show(); ui->SLEEP_BTNLabel_2->
                    >show(); ui->SleepButton->show();
                    characterL1Anim->setLoopAnimation(getLoopFrames());

                    // После стартовой анимации проверяем зарядку
                    updateChargingAnimation(batteryMonitor->isCharging());
                });
            });
            break;
        case 1: // Лаба 2
            characterL2Anim->playAnimation("intro");
            pciDevicesWidget->refreshDevices();
            break;
        default:
            characterAnim->playAnimation("intro"); // запасной вариант
            break;
    }
}

void MainWindow::updateBatteryUI()
{
    // ----- Источник питания -----
    ui->PowerSourceLabel->setText("Источник питания: " + batteryMonitor->powerSource());

    // ----- Уровень заряда -----
    batteryLevelLabel->setText("Заряд: " + QString::number(batteryMonitor->batteryLevel()) + "%");

    // ----- Время работы батареи -----
    QString msg;
    if (batteryMonitor->isCharging()) {
        if (batteryMonitor->batteryLevel() >= 100) {
            //msg = "Аккумулятор полностью заряжен";
        } else {
            int remainingSec = batteryMonitor->batteryFullLifeSeconds();
            if (remainingSec < 0) remainingSec = 0;
            int hours = remainingSec / 3600;
            int mins = (remainingSec % 3600) / 60;
            //msg = QString("Батарея заряжается").arg(hours).arg(mins);
        }
    } else {
        int remainingSec = batteryMonitor->batteryLifeSeconds();
        if (remainingSec < 0) remainingSec = 0;
        int hours = remainingSec / 3600;
        int mins = (remainingSec % 3600) / 60;
        msg = QString("До полной разрядки осталось: %1 ч %2 мин").arg(hours).arg(mins);
    }
    ui->BatteryTimeLabel->setText(msg);

    // ----- Режим энергосбережения -----
    QString savingMode = batteryMonitor->powerSavingMode();
    ui->PowerSavingModeLabel->setText("Режим энергосбережения: " + savingMode);

    // ----- Тип аккумулятора -----
    ui->BatteryTypeLabel->setText("Тип аккумулятора: " + batteryMonitor->batteryChemistry());
}

```

```

}

void MainWindow::updateChargingAnimation(bool charging)
{
    // Если мы не на странице 2, ничего не делаем
    if(ui->stackedWidget->currentIndex() != 1)
        return;

    if(charging) {
        // Зарядка подключена → IntroCharging → LoopCharging
        characterLlAnim->playSequence({"introCharging"}, [this]() {
            characterLlAnim->setLoopAnimation(getLoopChargingFrames());
        });
    } else {
        // Зарядка отключена → OutroCharging → обычный loop
        characterLlAnim->playSequence({"outroCharging"}, [this]() {
            characterLlAnim->setLoopAnimation(getLoopFrames());
        });
    }
}
}

```

Файл pcideviceswidget.h

```

#ifndef PCIDEVICESWIDGET_H
#define PCIDEVICESWIDGET_H
#include <QObject>
#include <QTableWidget>
#include <QHeaderView>
#include <QTableWidgetItem>
class PciDevicesWidget : public QObject
{
    Q_OBJECT

public:
    explicit PciDevicesWidget(QTableWidget *tableWidget, QObject *parent = nullptr);
    void refreshDevices();
private:
    QTableWidget *m_tableWidget;
    void setupTable();
    void populatePciDevices();
};
#endif // PCIDEVICESWIDGET_H

```

Файл pcideviceswidget.cpp

```

#include "pcideviceswidget.h"
#include <windows.h>
#include <setupapi.h>
#include <QDebug>
#include <QMessageBox>

PciDevicesWidget::PciDevicesWidget(QTableWidget *tableWidget, QObject *parent)
    : QObject(parent), m_tableWidget(tableWidget)
{
    setupTable();
}

void PciDevicesWidget::setupTable()
{
    if (!m_tableWidget) return;

    // Настраиваем таблицу
    m_tableWidget->setColumnCount(3);
    m_tableWidget->setHorizontalHeaderLabels(QStringList() << "Vendor ID" << "Device ID" << "Описание");

    // Настройка внешнего вида таблицы
    m_tableWidget->horizontalHeader()->setStretchLastSection(true);
    m_tableWidget->setEditTriggers(QAbstractItemView::NoEditTriggers);
    m_tableWidget->setSelectionBehavior(QAbstractItemView::SelectRows);
    m_tableWidget->setSelectionMode(QAbstractItemView::SingleSelection);
    m_tableWidget->setShowGrid(true);
    m_tableWidget->setStyleSheet("QTableWidget { gridline-color: #d0d0d0; }");

    // Устанавливаем начальные размеры колонок
    m_tableWidget->setColumnWidth(0, 100);
    m_tableWidget->setColumnWidth(1, 100);
    // Описание будет растягиваться, так как последняя колонка и setStretchLastSection(true)
}

void PciDevicesWidget::refreshDevices()
{
    if (!m_tableWidget) return;

    // Очищаем таблицу перед обновлением
    m_tableWidget->setRowCount(0);
    populatePciDevices();

    // Автоматически подгоняем размер столбцов под содержимое
    m_tableWidget->resizeColumnsToContents();
}

void PciDevicesWidget::populatePciDevices()
{
    if (!m_tableWidget) return;
    m_tableWidget->setRowCount(0);
}

```

```

HDEVINFO deviceInfoSet;
SP_DEVINFO_DATA deviceInfoData;
DWORD deviceIndex = 0;
DWORD error = 0;

// Получаем информацию о всех PCI устройствах
deviceInfoSet = SetupDiGetClassDevs(NULL, TEXT("PCI"), NULL, DIGCF_ALLCLASSES | DIGCF_PRESENT);

if (deviceInfoSet == INVALID_HANDLE_VALUE) {
    error = GetLastError();
    qDebug() << "Ошибка при получении списка PCI устройств. Код:" << error;
    QMessageBox::warning(nullptr, "Ошибка",
        QString("Не удалось получить информацию об устройствах PCI. Код ошибки: %1\nПроверьте права администратора.").arg(error));
    return;
}

qDebug() << "Начинаем перечисление PCI устройств...";

deviceInfoData.cbSize = sizeof(SP_DEVINFO_DATA);
int devicesFound = 0;

// Перебираем все устройства
while (SetupDiEnumDeviceInfo(deviceInfoSet, deviceIndex, &deviceInfoData)) {
    WCHAR hardwareID[1024];
    WCHAR deviceDesc[1024];
    DWORD requiredSize = 0;
    QString deviceDescStr = "Неизвестно";

    // Пытаемся получить описание устройства
    if (SetupDiGetDeviceRegistryPropertyW(deviceInfoSet, &deviceInfoData,
        SPDRP_DEVICEDESC, NULL,
        (PBYTE)deviceDesc, sizeof(deviceDesc) - sizeof(WCHAR),
        &requiredSize)) {
        deviceDesc[sizeof(deviceDesc)/sizeof(WCHAR) - 1] = L'\0';
        deviceDescStr = QString::fromWCharArray(deviceDesc);
    }

    // Затем получаем HardwareID
    if (SetupDiGetDeviceRegistryPropertyW(deviceInfoSet, &deviceInfoData,
        SPDRP_HARDWAREID, NULL,
        (PBYTE)hardwareID, sizeof(hardwareID) - sizeof(WCHAR),
        &requiredSize)) {
        hardwareID[sizeof(hardwareID)/sizeof(WCHAR) - 1] = L'\0';
        QString hardwareIDStr = QString::fromWCharArray(hardwareID).toUpper();

        // Ищем VendorID и DeviceID в строке Hardware ID
        if (hardwareIDStr.contains("PCI\\VEN_", Qt::CaseInsensitive)) {
            QStringList parts = hardwareIDStr.split("&");
            QString vendorID, deviceID;

            for (const QString &part : parts) {
                QString cleanPart = part.trimmed();
                if (cleanPart.startsWith("PCI\\VEN_", Qt::CaseInsensitive)) {
                    // Извлекаем 4 символа после VEN_
                    QString venPart = cleanPart.mid(8);
                    if (venPart.length() >= 4) {
                        vendorID = venPart.left(4);
                    }
                } else if (cleanPart.startsWith("DEV_", Qt::CaseInsensitive)) {
                    // Извлекаем 4 символа после DEV_
                    QString devPart = cleanPart.mid(4);
                    if (devPart.length() >= 4) {
                        deviceID = devPart.left(4);
                    }
                } else if (cleanPart.startsWith("SUBSYS_", Qt::CaseInsensitive)) {
                    // Пропускаем SUBSYS
                    continue;
                }
            }

            // Если нашли оба ID, добавляем в таблицу
            if (!vendorID.isEmpty() && !deviceID.isEmpty()) {
                int row = m_tableWidget->rowCount();
                m_tableWidget->insertRow(row);

                m_tableWidget->setItem(row, 0, new QTableWidgetItem(vendorID.toUpper()));
                m_tableWidget->setItem(row, 1, new QTableWidgetItem(deviceID.toUpper()));
                m_tableWidget->setItem(row, 2, new QTableWidgetItem(deviceDescStr));
                devicesFound++;

                qDebug() << "Добавлено устройство: VendorID:" << vendorID << "DeviceID:" << deviceID << "Описание:" <<
deviceDescStr;
            } else {
                qDebug() << "Не удалось извлечь ID из:" << hardwareIDStr;
            }
        } else {
            error = GetLastError();
            if (error != ERROR_INVALID_DATA) {
                qDebug() << "Ошибка получения HardwareID для устройства" << deviceIndex << "Код ошибки:" << error;
            }
        }

        deviceIndex++;
        ZeroMemory(&deviceInfoData, sizeof(SP_DEVINFO_DATA));
        deviceInfoData.cbSize = sizeof(SP_DEVINFO_DATA);
    }

    error = GetLastError();
    if (error != ERROR_NO_MORE_ITEMS) {
        qDebug() << "Ошибка при перечислении устройств. Код:" << error;
    }
}

```

```

SetupDiDestroyDeviceInfoList(deviceInfoSet);

qDebug() << "Перечисление завершено. Всего найдено PCI устройств:" << devicesFound;

if (devicesFound == 0) {
    QMessageBox::information(nullptr, "Информация",
        "PCI устройства не найдены. Возможные причины:\n"
        "1. Отсутствуют права администратора\n"
        "2. Проблема с драйверами PCI\n"
        "3. Аппаратная проблема\n"
        "4. Попробуйте запустить программу от имени администратора");
}
}

```

2.2 Результат выполнения программы

Основной графический интерфейс представлен на рисунке 2, также на нём изображено выполнение основного задания лабораторной работы.

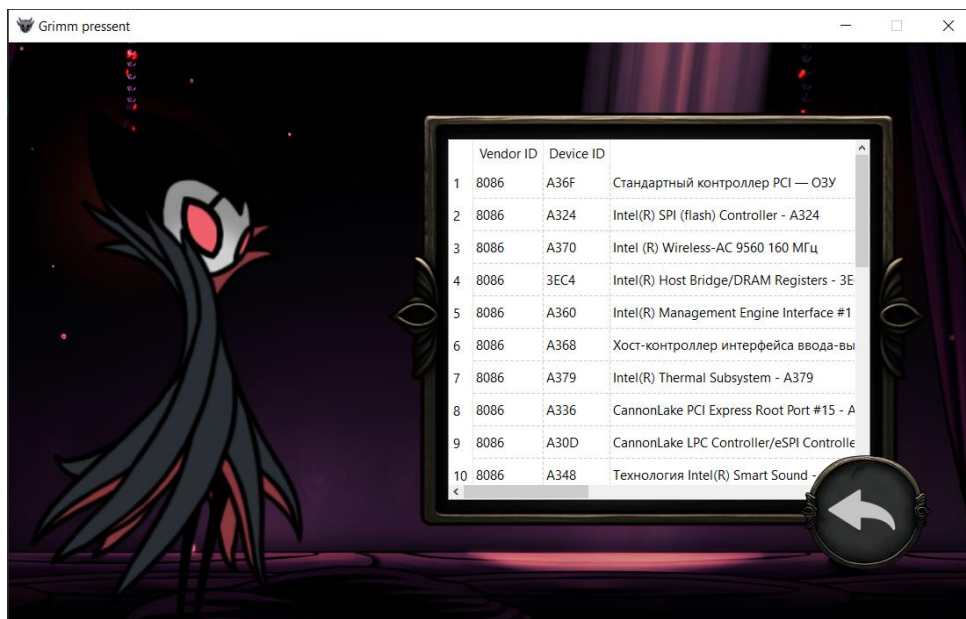


Рисунок 2 – Результат выполнения программы

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы успешно реализована поставленная задача — исследование архитектуры PCI-шины и разработка программного модуля для обнаружения подключенных устройств с использованием Windows SetupAPI. Созданное приложение способно в реальном времени отображать список всех PCI-устройств системы, их идентификаторы VendorID и DeviceID, а также предоставляет понятные пользователю названия производителей и типы оборудования.

В качестве практического результата разработан графический интерфейс, интегрированный в общее приложение с анимационными элементами, что демонстрирует эффективное сочетание низкоуровневых системных функций с современными возможностями визуализации.

Изучение методов работы с PCI-устройствами остается актуальным направлением, поскольку полученные знания находят применение при создании диагностических утилит, систем мониторинга оборудования, разработке драйверов устройств, а также являются важной основой для дальнейшего освоения системного программирования и анализа компьютерных архитектур.